Defensoria Pública
BAHIA

Designações do Júri

Arquitetura, modelo de segurança e relatório de testes de intrusão

Documento técnico · outubro de 2026

Arquitetura

Navegador
SPA estática · HTML/CSS/JS vanilla, sem build

→

Netlify
CDN · headers de segurança (CSP, XFO, nosniff)

→

Supabase
PostgREST · PostgreSQL 17 · Edge Functions (Deno)

- **Frontend**: SPA com roteamento por hash, render por template strings com escape centralizado (`esc()`).
- **Dados**: modelo documento-chave em uma tabela `docs(colecao, chave, dados jsonb)`; PK composta.
- **Sincronização**: polling de 10 s por RPC `versao_docs()` (count + max(updated_at)); refetch paginado (Range) só quando a assinatura muda.
- **Integrações**: Google Routes/Places API (rotação de chaves no cliente) e DataJud/CNJ via Edge Function proxy.

Autenticação e sessão

- Credenciais em `usuarios(login, senha_hash)` com **bcrypt** (`pgcrypto.crypt`, custo 10). Sem signup nem reset de senha (superfície reduzida).
- Login via RPC `entrar(login, senha)` → token de sessão **UUIDv4** (122 bits de entropia) em `sessoes`.
- Cliente envia o token no header `x-sessao`; o backend lê via `current_setting('request.headers')`.
- **Expiração** de 30 dias; logout revoga a sessão no servidor (`DELETE`).
- **Rate limiting** por conta: 10 falhas / 15 min → bloqueio temporário (tabela `tentativas`). Sem `pg_sleep` (evita exaustão do pool de conexões).
- Comparação de senha dentro do banco; mesma resposta (`null`) para usuário inexistente e senha errada.

Autorização: Row Level Security

- RLS habilitado em todas as tabelas. Políticas de `docs` (SELECT/INSERT/UPDATE/DELETE) exigem `privado.sessao_valida()`, avaliada uma vez por statement (`(select …)` → initPlan).
- Tabelas sensíveis (`usuarios`, `sessoes`, `tentativas`, `acesso`): `REVOKE ALL` de `anon/authenticated` + política **RESTRICTIVE** `USING(false)` (defesa em profundidade).
- Funções **SECURITY DEFINER** isoladas no schema `privado`, **não exposto** pelo PostgREST (só `public` e `graphql_public`); `search_path` fixado.
- Funções em `public` são **SECURITY INVOKER** (wrappers finos). Supabase Security Advisor: **0 warnings**.
- A chave *publishable* no frontend é pública por design; não concede acesso a dados sem sessão válida.

Segurança no cliente e na borda

- **XSS**: todo dado vindo do banco passa por `esc()`; IDs validados por regex `^[A-Za-z0-9_-]{1,64}$`; datas por `^\d{4}-\d{2}-\d{2}$`; enum de status por allowlist. Sanitização aplicada na carga (rede, localStorage e import de backup).
- **CSP**: `default-src 'self'; script-src 'self'` (sem inline/eval), `connect-src` restrito (Supabase, Google, OSRM), `frame-ancestors 'none'`, `base-uri 'self'`, `form-action 'self'`.
- **Clickjacking**: `X-Frame-Options: DENY`. **MIME sniffing**: `nosniff`. Referrer-Policy restritiva.
- Dependências de terceiros (jsPDF, autotable) **vendorizadas** e servidas da própria origem (sem CDN em runtime).
- Edge Function DataJud valida a sessão via RPC antes de consultar o CNJ; CORS controlado; entrada validada (CNJ 20 dígitos, mapa de tribunais).

Modelo de ameaças

Ator	Capacidade assumida	Objetivo
Atacante externo	URL do projeto + chave publishable (lidas do bundle); HTTP arbitrário	Ler/alterar dados, enumerar schema, negar serviço
Usuário autenticado malicioso	Sessão válida, escrita em `docs`	XSS armazenado contra colegas, roubo de sessão
Força bruta	Logins previsíveis	Adivinhar senha
Abuso de recurso	Chamadas em massa	Esgotar conexões / cotas de API

Mapeamento OWASP Top 10 (2021)

Categoria	Controle aplicado
A01 Broken Access Control	RLS em todas as tabelas, schema privado, políticas restritivas
A02 Cryptographic Failures	bcrypt; TLS ponta a ponta (Netlify/Supabase)
A03 Injection	Queries parametrizadas (PostgREST/RPC), escape de HTML, validação por allowlist
A04 Insecure Design	Sem signup/reset; mínimo privilégio; superfície reduzida
A05 Security Misconfiguration	CSP, XFO, nosniff; Advisor sem warnings
A07 Identification & Auth Failures	Rate limit, expiração e revogação de sessão
A08 Software & Data Integrity	Libs vendorizadas; sanitização de backups importados

Metodologia dos testes de intrusão

- **Abordagem**: caixa-cinza (gray-box): conhecimento do código, mas ataques apenas com o que é público (URL + chave publishable).
- **Camada de banco**: 15 casos executados com `SET ROLE anon` e headers simulados (`request.headers`), replicando exatamente o contexto do PostgREST.
- **Camada HTTP externa**: 12 requisições reais à API pública (REST, RPC, Edge Function, GraphQL), originadas fora da aplicação.
- **Camada cliente**: injeção de payloads XSS armazenados (`<img onerror>`, quebra de atributo em IDs, datas malformadas) e varredura de todas as telas com Chromium headless (Playwright).
- **Critério de aprovação**: nenhuma linha lida/alterada sem sessão; nenhum script injetado executado; integridade confirmada por contagem antes/depois.

Casos de teste: API (externo)

#	Vetor	Resposta	
E1	GET /rest/v1/docs sem sessão	200 [] (RLS filtra)	PASS
E2/E3	GET usuarios / sessoes	401 / 42501 permission denied	PASS
E4	x-sessao com UUID forjado	200 []	PASS
E5	POST docs (insert)	401 RLS violation	PASS
E6	DELETE docs?colecao=neq.x	204, 0 linhas (verificado)	PASS
E7	Accept-Profile: privado	406 PGRST106 schema não exposto	PASS
E8/E9	RPC entrar: senha errada / SQLi	200 null	PASS
E10	Edge Function sem sessão	401	PASS
E11/E12	OpenAPI / GraphQL introspection	401 / extensão desativada	PASS

Casos de teste: banco e cliente

#	Vetor	Resultado	
A1–A3	SELECT em usuarios, sessoes, tentativas como anon	permission denied	PASS
A5–A7	INSERT/DELETE em docs; sessão falsa	RLS bloqueia / 0 linhas	PASS
A8/A9	SQL injection em login e senha	negado (parâmetros tipados)	PASS
A12/A13	Criar usuário / trocar senha alheia	permission denied	PASS
A14	DDL (CREATE TABLE) como anon	permission denied for schema	PASS
X1	XSS armazenado em todas as telas	4 execuções → corrigido → 0	FIXED

Achados e correções

Achado	Severidade	Correção
`pg_sleep(1)` no login permitia exaustão do pool de conexões (DoS)	Alta	Removido; substituído por rate limit por conta
Força bruta sem limite	Alta	10 falhas / 15 min
Funções SECURITY DEFINER expostas no schema público	Média	Movidas para schema privado; wrappers INVOKER
Sessões sem expiração	Média	TTL de 30 dias
XSS armazenado (IDs, datas, chip de data no painel)	Média	Escape + validação por allowlist + CSP sem inline

Riscos residuais e recomendações

- **Credenciais fracas e compartilhadas**: adotar senhas únicas e fortes por usuário; considerar 2FA ou SSO institucional.
- **Lockout como vetor de DoS direcionado**: um atacante pode bloquear uma conta por 15 min; mitigável com limite por IP no gateway.
- **Mensagens de erro** do PostgREST revelam nomes de tabelas (baixo impacto, sem dados).
- **Chaves do Google** visíveis a usuários autenticados; restringir por HTTP referrer e por API no Google Cloud.
- **Disponibilidade**: plano gratuito pausa após 7 dias de inatividade; considerar plano pago ou keep-alive.
- **Auditoria**: adicionar log de alterações (quem/quando) por documento.